

MODULE -3

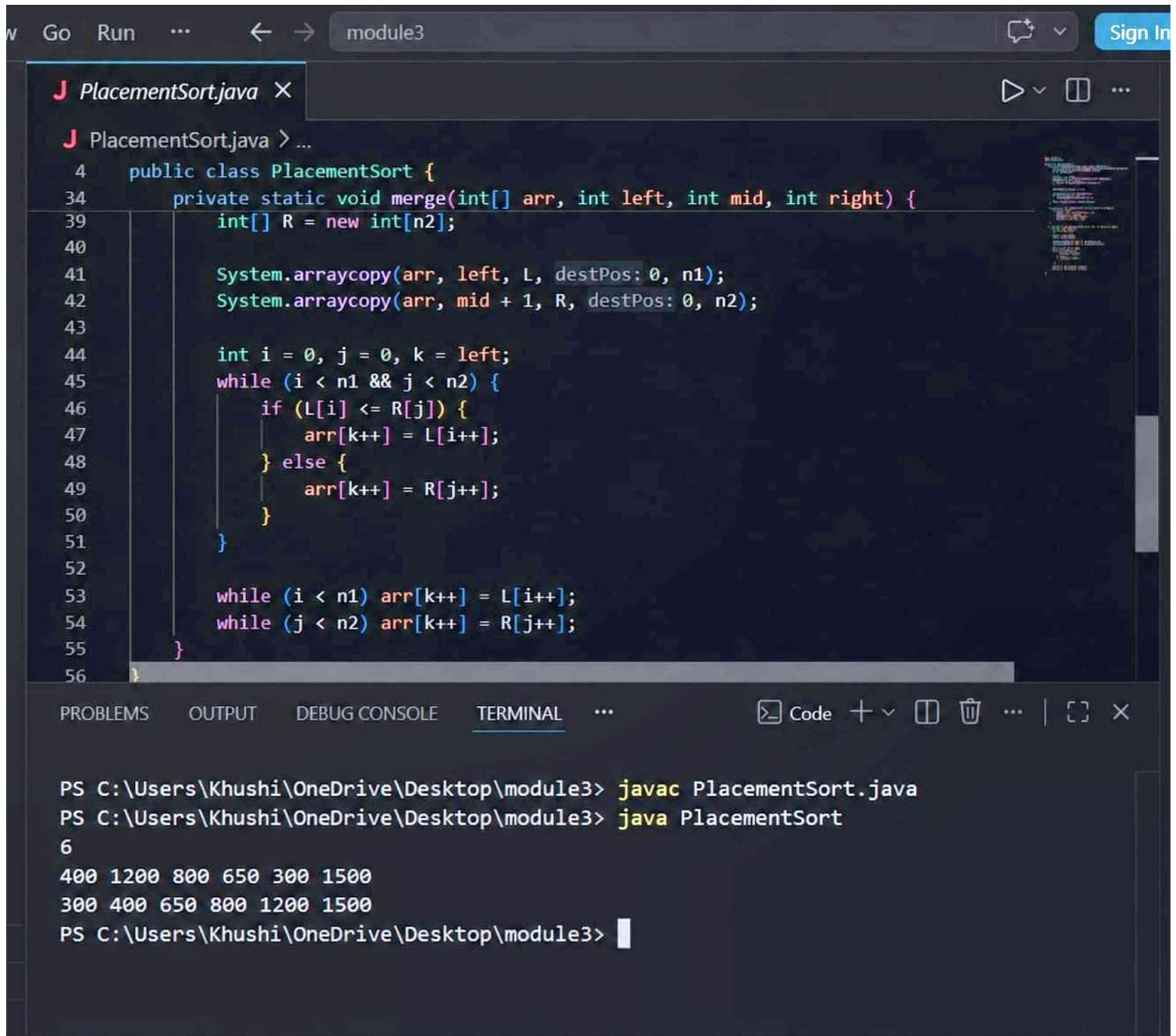
Name - khushi kumari

Course - B.tech

Branch - ALML

Topic - Merge Sort

1.(easy)



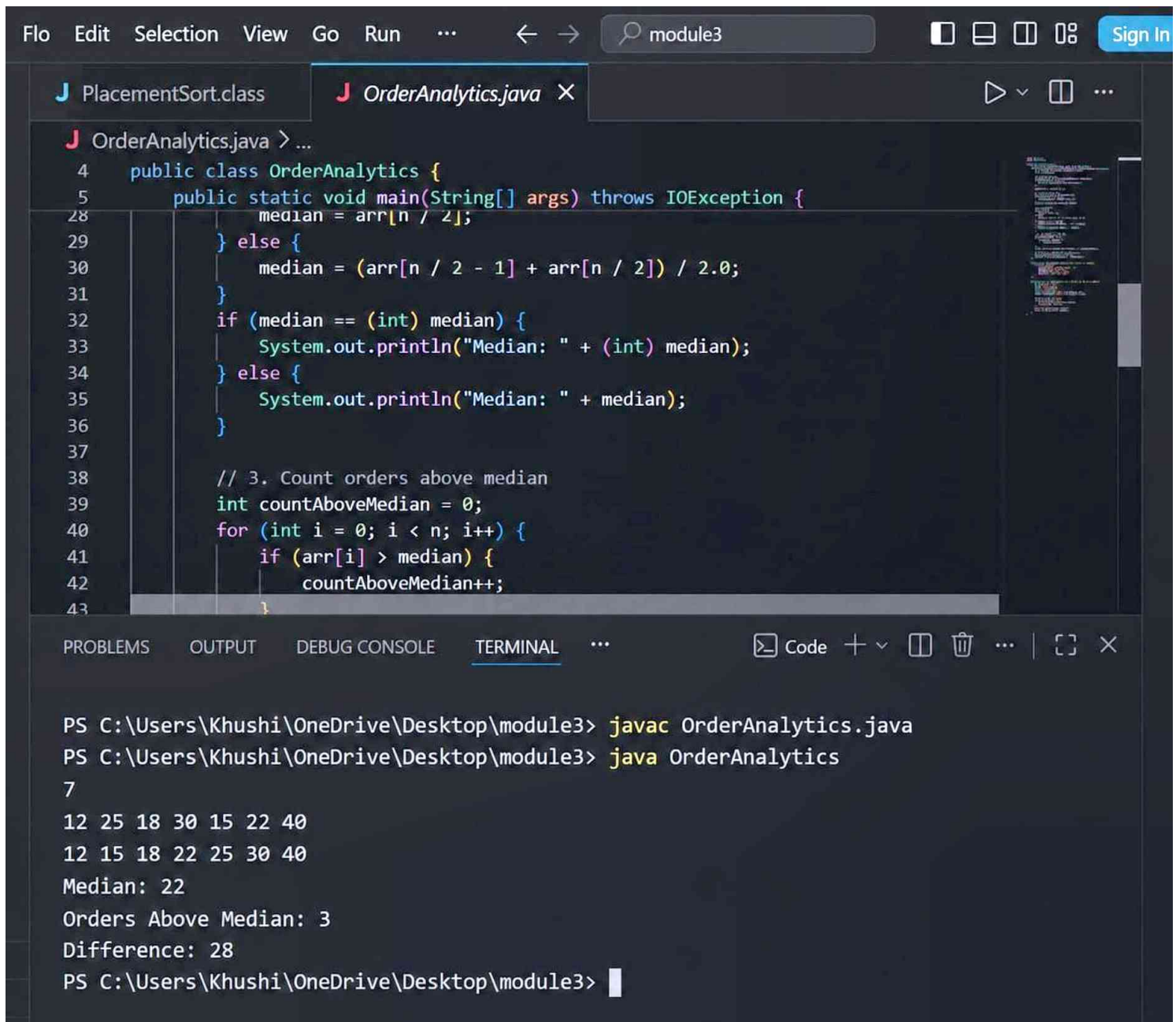
The screenshot displays an IDE window titled 'module3' with a file named 'PlacementSort.java'. The code implements a merge sort algorithm. It defines a 'merge' method that takes an array 'arr', and indices 'left', 'mid', and 'right'. It creates a temporary array 'R' and copies the elements from 'arr' into it. Then, it merges the two sorted halves back into 'arr' using a while loop. Finally, it copies the sorted elements back from 'R' to 'arr'.

```
4 public class PlacementSort {
34 private static void merge(int[] arr, int left, int mid, int right) {
39     int[] R = new int[n2];
40
41     System.arraycopy(arr, left, L, destPos: 0, n1);
42     System.arraycopy(arr, mid + 1, R, destPos: 0, n2);
43
44     int i = 0, j = 0, k = left;
45     while (i < n1 && j < n2) {
46         if (L[i] <= R[j]) {
47             arr[k++] = L[i++];
48         } else {
49             arr[k++] = R[j++];
50         }
51     }
52
53     while (i < n1) arr[k++] = L[i++];
54     while (j < n2) arr[k++] = R[j++];
55 }
56 }
```

The terminal output shows the execution of the program, displaying the initial array and the sorted array.

```
PS C:\Users\Khushi\OneDrive\Desktop\module3> javac PlacementSort.java
PS C:\Users\Khushi\OneDrive\Desktop\module3> java PlacementSort
6
400 1200 800 650 300 1500
300 400 650 800 1200 1500
PS C:\Users\Khushi\OneDrive\Desktop\module3>
```

Topic - Merge Sort 2.(Medium)



The screenshot shows an IDE with two tabs: `PlacementSort.class` and `OrderAnalytics.java`. The `OrderAnalytics.java` tab is active, displaying the following code:

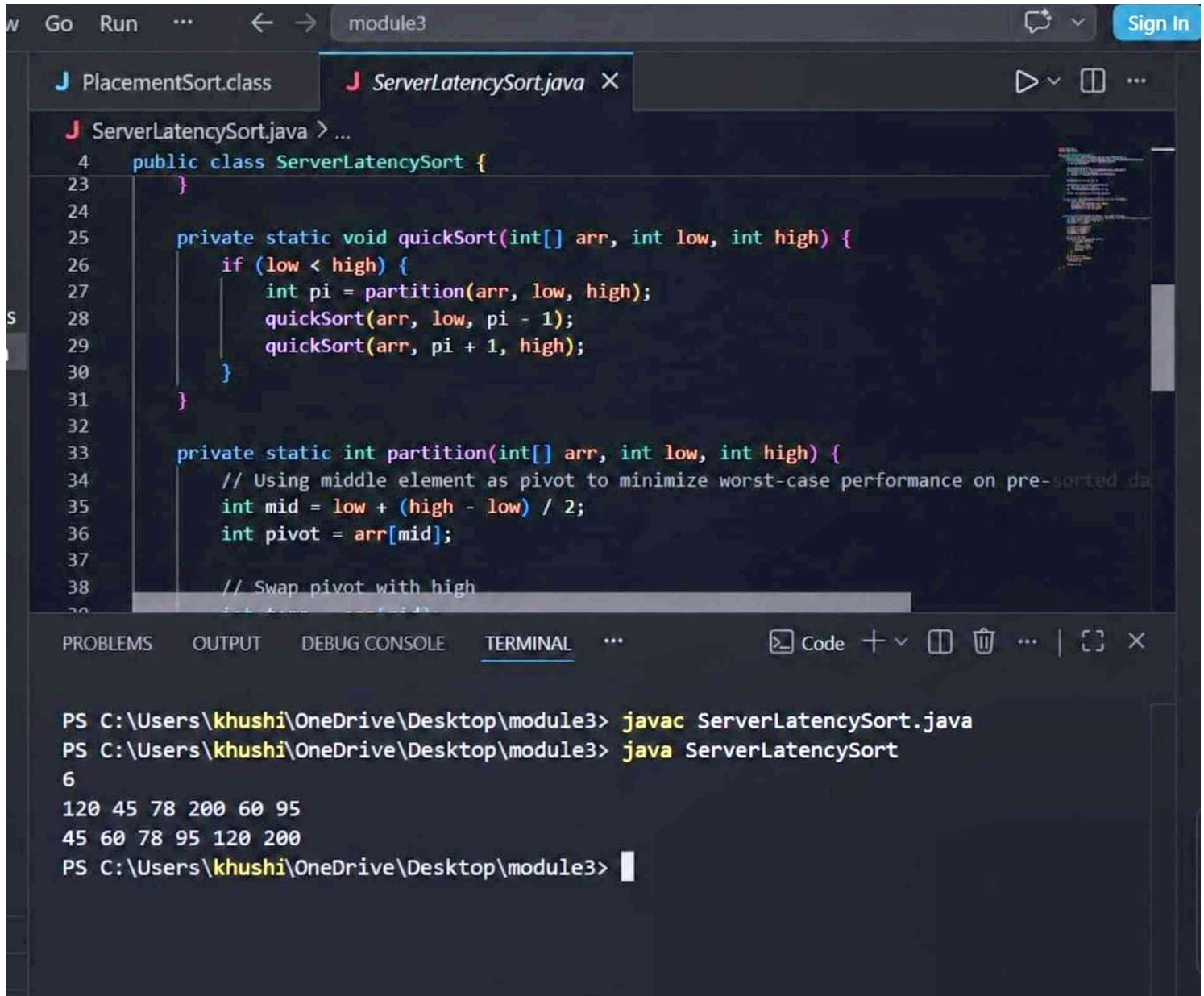
```
4 public class OrderAnalytics {
5     public static void main(String[] args) throws IOException {
28         median = arr[n / 2];
29     } else {
30         median = (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
31     }
32     if (median == (int) median) {
33         System.out.println("Median: " + (int) median);
34     } else {
35         System.out.println("Median: " + median);
36     }
37
38     // 3. Count orders above median
39     int countAboveMedian = 0;
40     for (int i = 0; i < n; i++) {
41         if (arr[i] > median) {
42             countAboveMedian++;
43         }
44     }
```

The terminal output shows the execution of the program:

```
PS C:\Users\Khushi\OneDrive\Desktop\module3> javac OrderAnalytics.java
PS C:\Users\Khushi\OneDrive\Desktop\module3> java OrderAnalytics
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\Khushi\OneDrive\Desktop\module3>
```

Topic - Quick Sort

1.(Easy)



```
W Go Run ... < -> module3 Sign In
J PlacementSort.class J ServerLatencySort.java X
J ServerLatencySort.java > ...
4 public class ServerLatencySort {
23 }
24
25 private static void quickSort(int[] arr, int low, int high) {
26     if (low < high) {
27         int pi = partition(arr, low, high);
28         quickSort(arr, low, pi - 1);
29         quickSort(arr, pi + 1, high);
30     }
31 }
32
33 private static int partition(int[] arr, int low, int high) {
34     // Using middle element as pivot to minimize worst-case performance on pre-sorted data
35     int mid = low + (high - low) / 2;
36     int pivot = arr[mid];
37
38     // Swap pivot with high
39     int temp = arr[high];
40     arr[high] = arr[mid];
41     arr[mid] = temp;
42
43     int i = low;
44     for (int j = low; j < high; j++) {
45         if (arr[j] < pivot) {
46             int temp = arr[i];
47             arr[i] = arr[j];
48             arr[j] = temp;
49             i++;
50         }
51     }
52     int temp = arr[i];
53     arr[i] = arr[high];
54     arr[high] = temp;
55     return i;
56 }
57 }
58
59 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL ... Code + - [] [] ... | [] X
PS C:\Users\khushi\OneDrive\Desktop\module3> javac ServerLatencySort.java
PS C:\Users\khushi\OneDrive\Desktop\module3> java ServerLatencySort
6
120 45 78 200 60 95
45 60 78 95 120 200
PS C:\Users\khushi\OneDrive\Desktop\module3>
```

Topic - Quick Sort

2.(Medium)

```
view Go Run ... < -> module3 Sign In
```

```
TradeAnalytics.class X
```

```
7 public class TradeAnalytics {
8     public TradeAnalytics() {
9     }
10
11     public static void main(String[] var0) throws IOException {
12         BufferedReader var1 = new BufferedReader(new InputStreamReader(System.in));
13         int var2 = Integer.parseInt(var1.readLine().trim());
14         if (var2 >= 5) {
15             long[] var3 = new long[var2];
16             long var4 = 0L;
17             StringTokenizer var6 = new StringTokenizer(var1.readLine());
18
19             for(int var7 = 0; var7 < var2; ++var7) {
20                 var3[var7] = Long.parseLong(var6.nextToken());
21                 var4 += var3[var7];
22             }
23
24             double var16 = (double)var4 / (double)var2;
25             quickSortDescending(var3, 0, var2 - 1);
26         }
27     }
28 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... Code + - [] [X]
```

```
PS C:\Users\khushi\OneDrive\Desktop\module3> javac TradeAnalytics.java
PS C:\Users\khushi\OneDrive\Desktop\module3> java TradeAnalytics
8
3000 2500 2000 1500 1200 900 700 500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\khushi\OneDrive\Desktop\module3>
```


1.(Easy)

```

1 LeaderboardSort.java > LeaderboardSort
4     public class LeaderboardSort {
5         public static void main(String[] args) throws IOException {
8             if (n == 0) return;
9
10            int[] arr = new int[n];
11            StringTokenizer st = new StringTokenizer(br.readLine());
12            for (int i = 0; i < n; i++) {
13                arr[i] = Integer.parseInt(st.nextToken());
14            }
15
16            heapSort(arr);
17
18            StringBuilder sb = new StringBuilder();
19            for (int i = 0; i < n; i++) {
20                sb.append(arr[i]).append(" ");
21            }
22            System.out.println(sb.toString().trim());

```

Topic - Heap Sort

2.(Medium)

```
Go Run ... < > module3 Sign In
```

```
J HospitalMetrics.java X
```

```
J HospitalMetrics.java > HospitalMetrics
```

```
4 public class HospitalMetrics {
5     public static void main(String[] args) throws IOException {
6
7         if (n == 0) return;
8
9         int[] arr = new int[n];
10        double sum = 0;
11        StringTokenizer st = new StringTokenizer(br.readLine());
12        for (int i = 0; i < n; i++) {
13            arr[i] = Integer.parseInt(st.nextToken());
14            sum += arr[i];
15        }
16
17        heapSort(arr);
18
19        // 1. Display sorted response times
20        StringBuilder sb = new StringBuilder();
21        for (int i = 0; i < n; i++) {
22            sb.append(arr[i] + " ");
23        }
24        System.out.println(sb);
25
26        // 2. Find fastest and slowest response times
27        int fastest = arr[0];
28        int slowest = arr[0];
29        for (int i = 1; i < n; i++) {
30            if (arr[i] < fastest) fastest = arr[i];
31            if (arr[i] > slowest) slowest = arr[i];
32        }
33
34        // 3. Find number of cases faster than average
35        double average = sum / n;
36        int faster = 0;
37        for (int i = 0; i < n; i++) {
38            if (arr[i] < average) faster++;
39        }
40
41        // 4. Calculate percentage of cases faster than average
42        double percentage = (faster * 100) / n;
43
44        // 5. Print results
45        System.out.println("Fastest: " + fastest);
46        System.out.println("Slowest: " + slowest);
47        System.out.println("Average: " + average);
48        System.out.println("Cases Faster Than Average: " + faster);
49        System.out.println("Percentage: " + percentage + "%");
50    }
51}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS OPENSHIFT TERMINAL Code + - [] [] ... | [] X
```

```
PS C:\Users\khushi\OneDrive\Desktop\module3> javac HospitalMetrics.java
PS C:\Users\khushi\OneDrive\Desktop\module3> java HospitalMetrics
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\khushi\OneDrive\Desktop\module3>
```

```
In 4 Col 29 (15 selected) Spaces: 4 UTF-8 CRLF {} Java
```